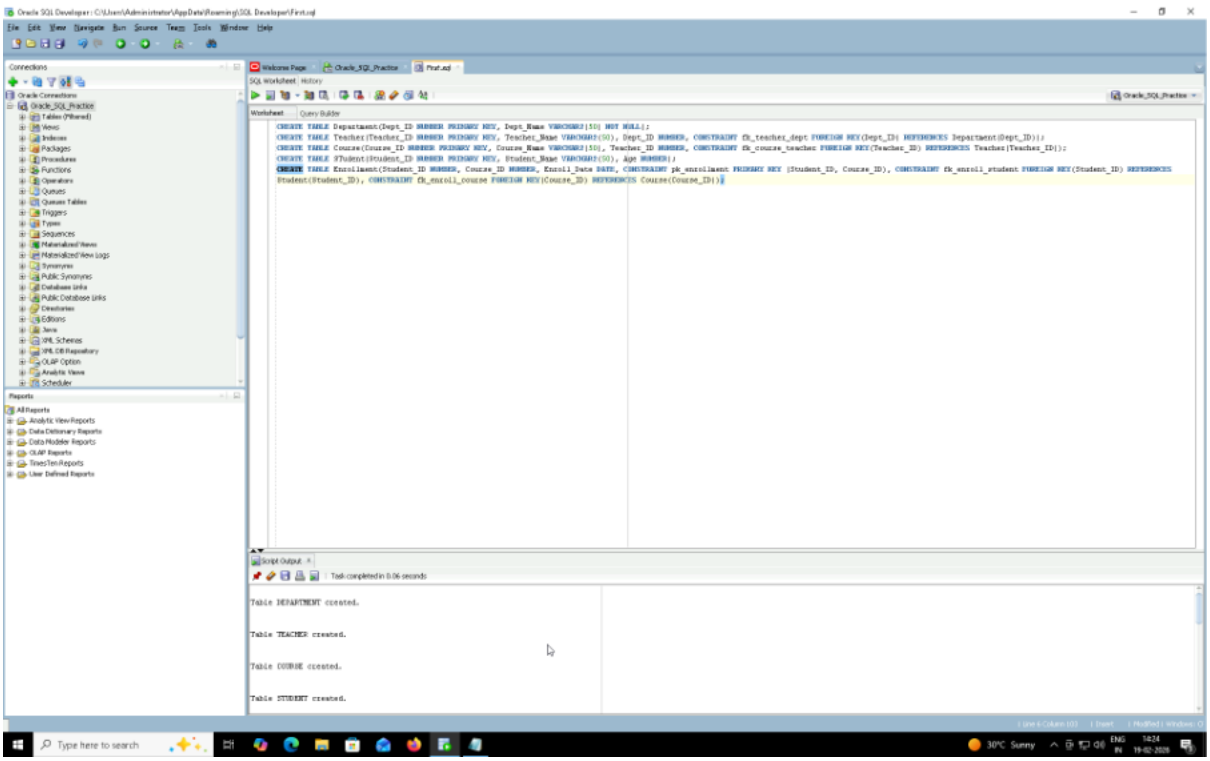
TOPIC – ORABCLE DBA

Assignment 1: Creating tables with primary key and foreign key using normalization theories

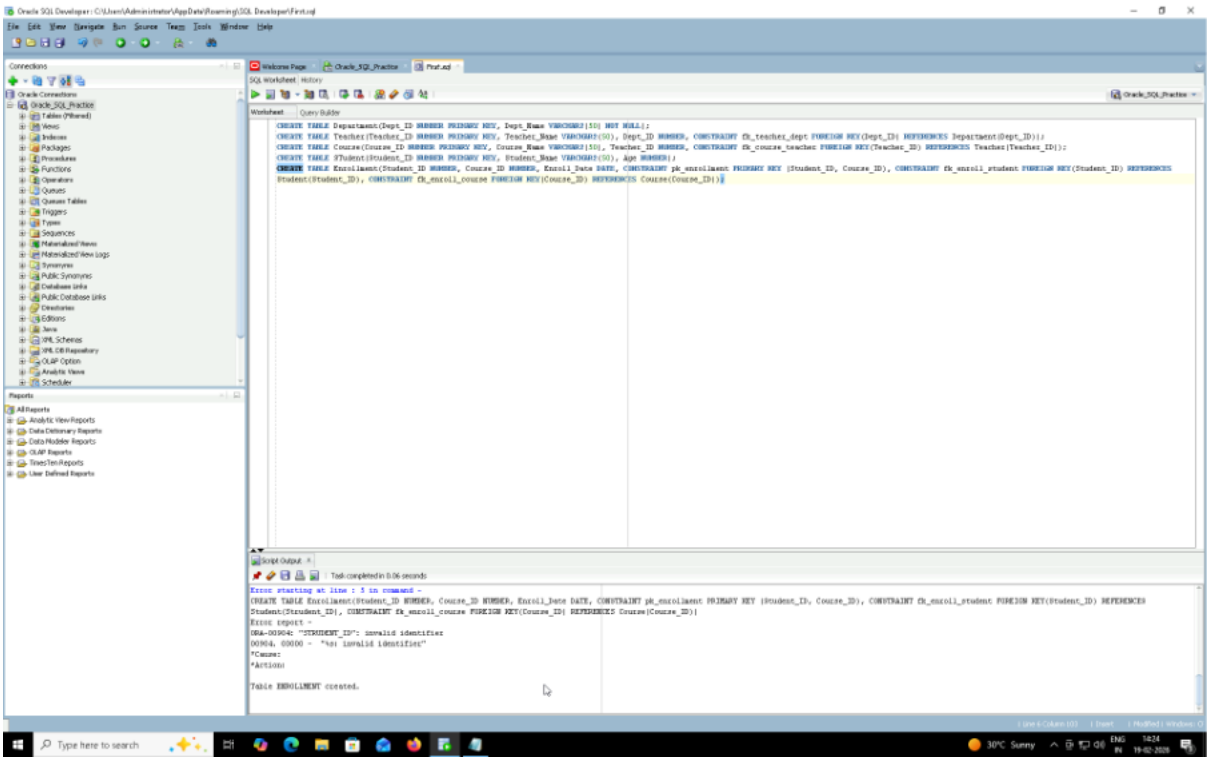
- Case Study 1: Student-Course Enrolment System
    - Objective: Design a database to manage student enrolment in courses.
    - Requirements:
        - A Student can enrol in multiple Courses.
        - Each Course is taught by a teacher.
        - A Teacher belongs to a specific Department. Step 1: Identify Tables
        - Based on the requirements, we identify the following entities:
            - Student
            - Course
            - Teacher
            - Department
            - Enrolment (Relationship between Student and Course) Step 2: Apply
        - Normalization
            - 1NF: Remove redundant data by separating related data into distinct tables.
            - Separate students, courses, teachers, and departments into their own tables.
            - 2NF: Ensure no partial dependencies.
            - Each table has a primary key that uniquely identifies each record.
            - 3NF: Remove transitive dependencies.

We will be implementing this: -

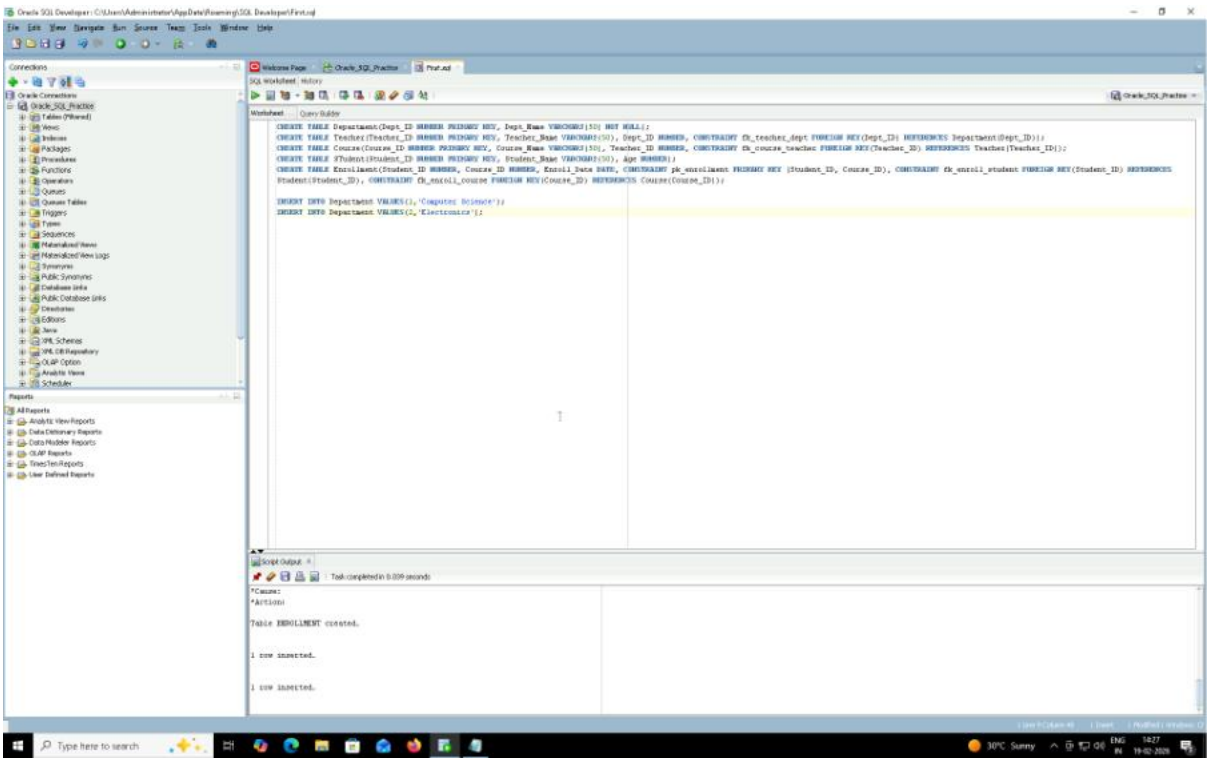
- Student <-> Course (Many-to-Many)
- Teacher <-> Course (One-to-Many)
- Department <-> Teacher (One-to-Many)



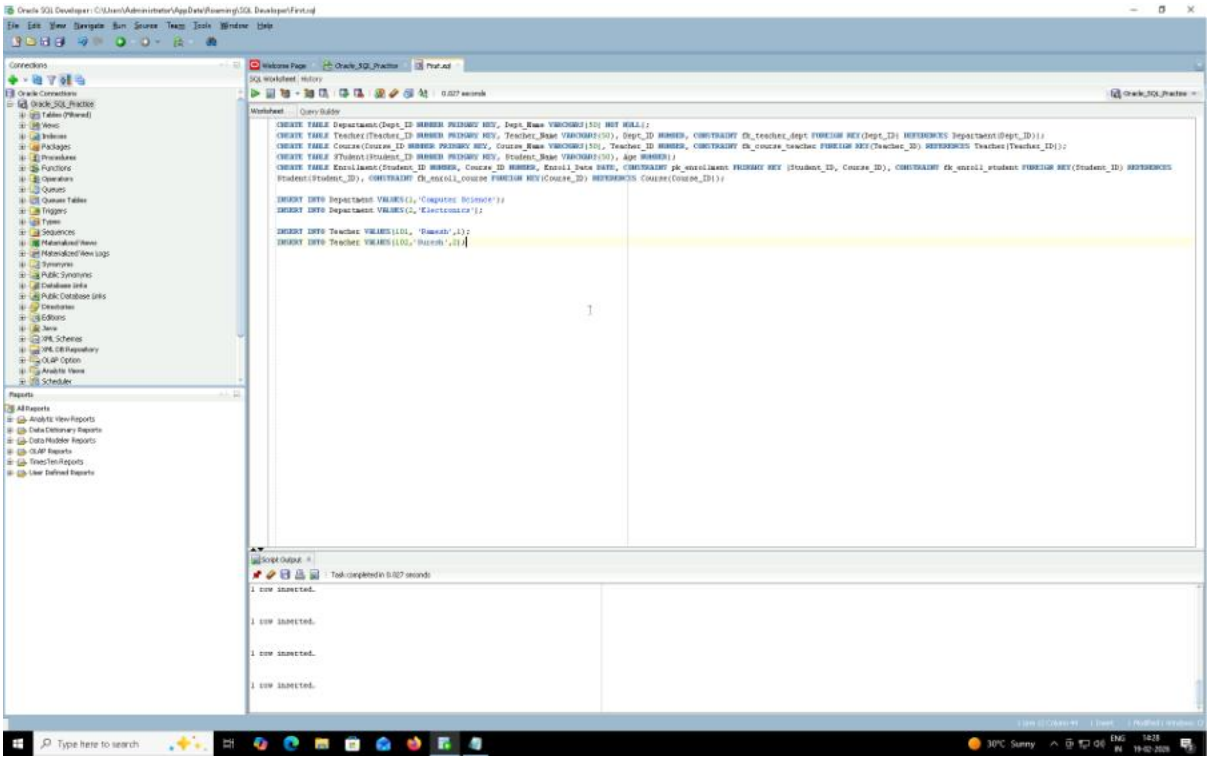
All tables have been created successfully.



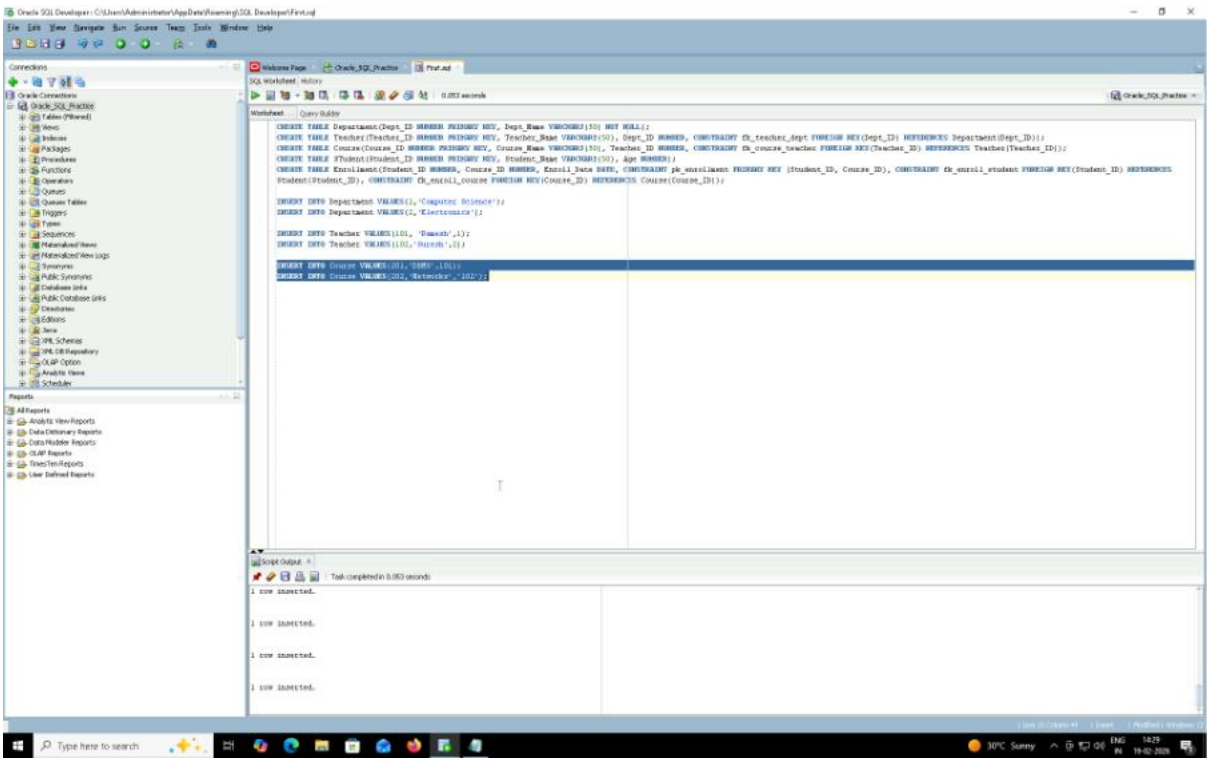
Enrolment table resolves many-to-many relationship between Student and Course using composite primary key.



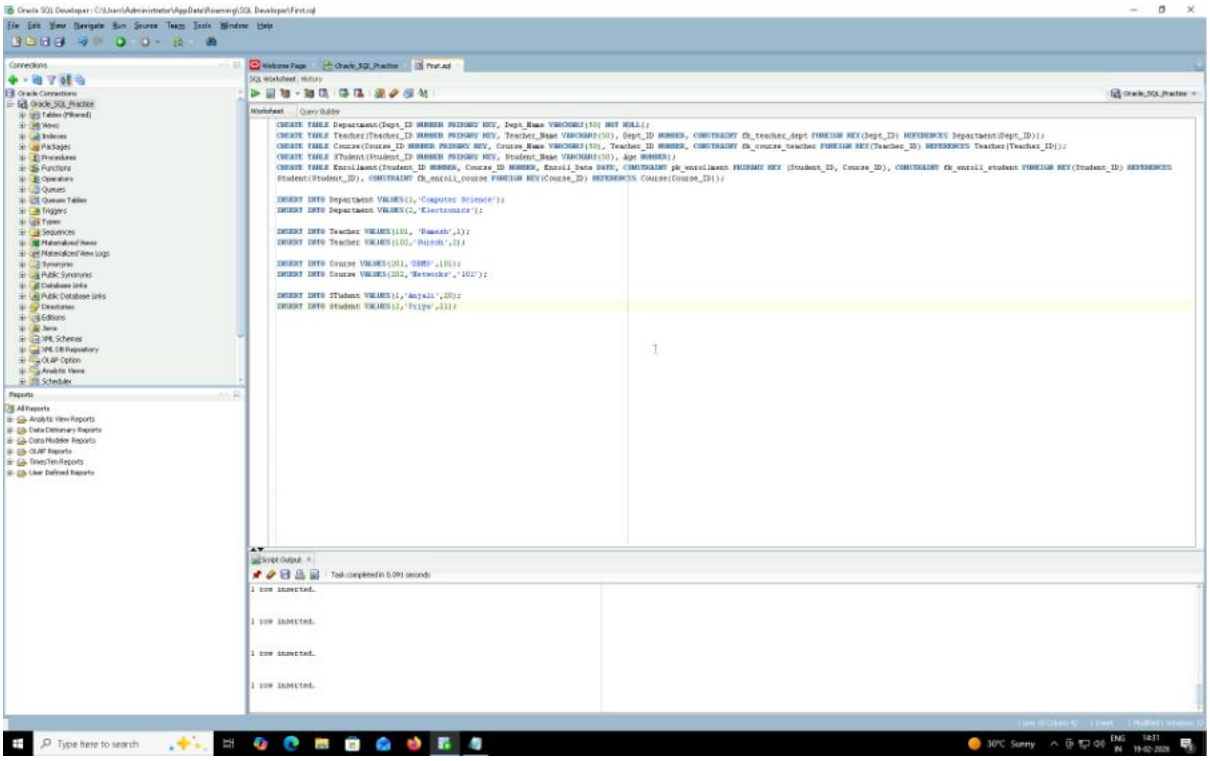
Records have been inserted into Department Table.



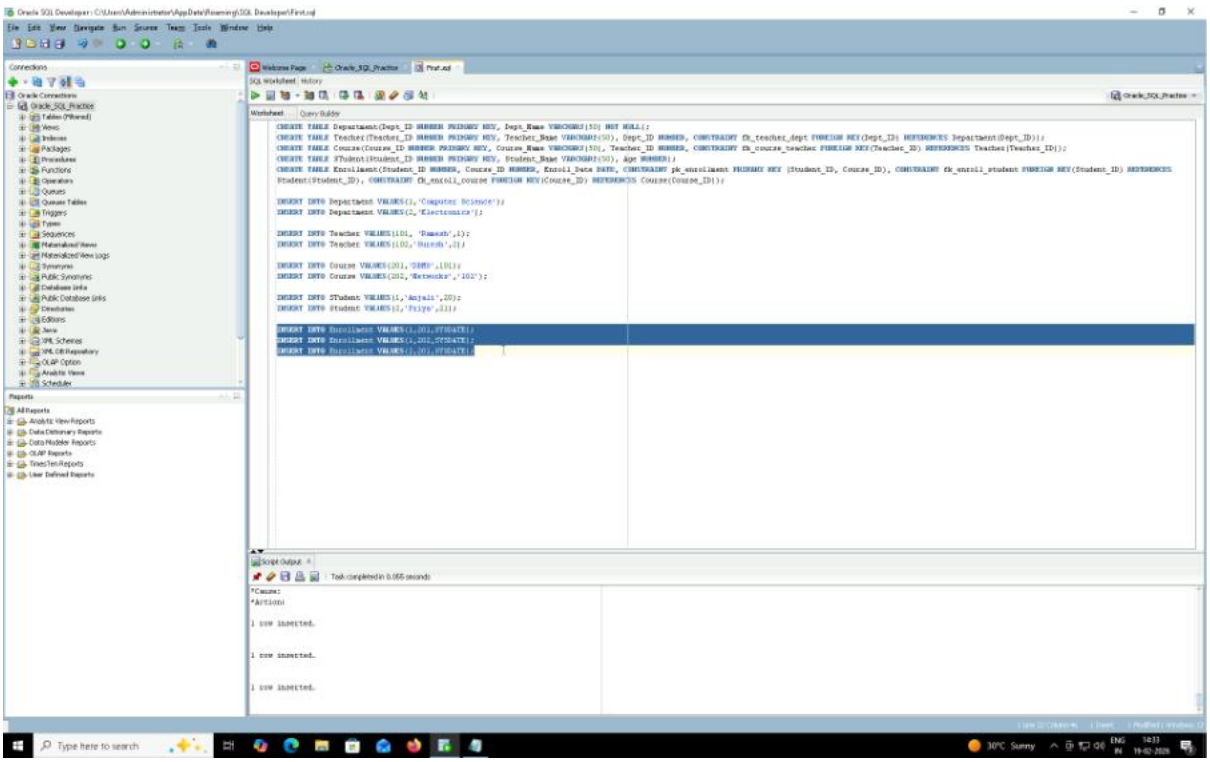
Records have been inserted into Teacher Table.



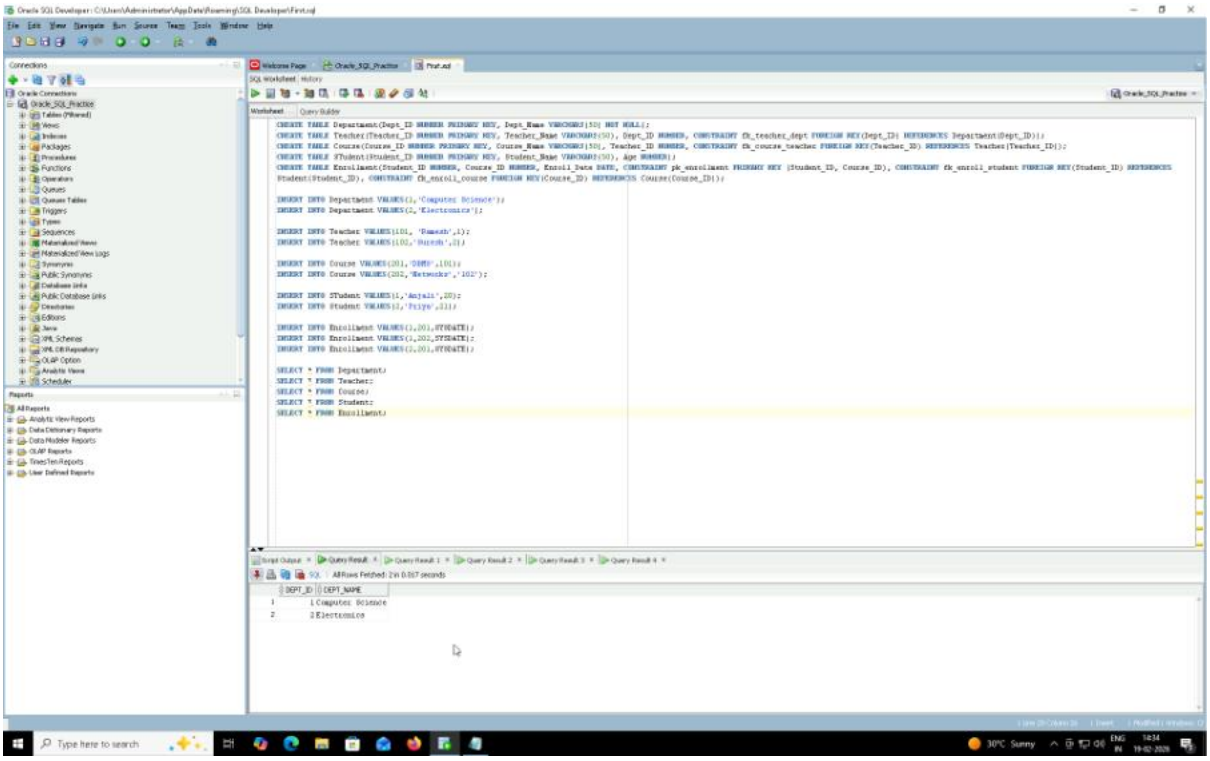
Records have been inserted into Courses Table.

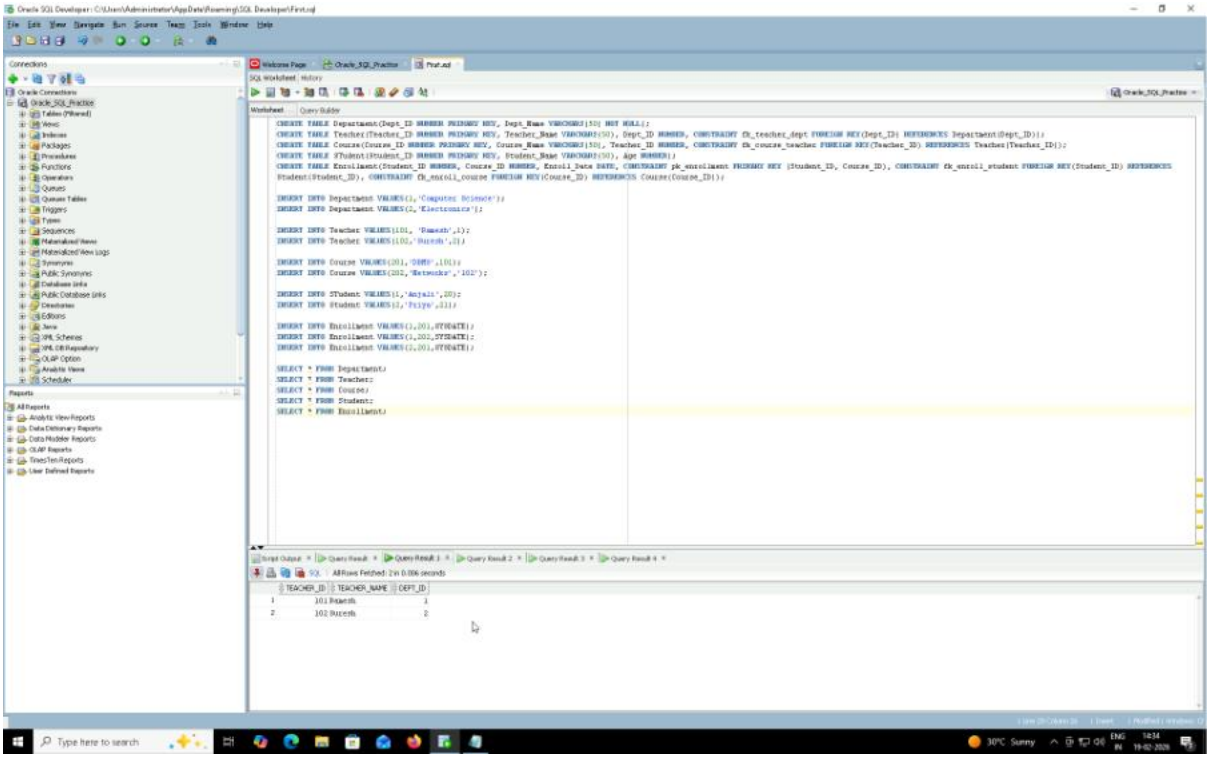


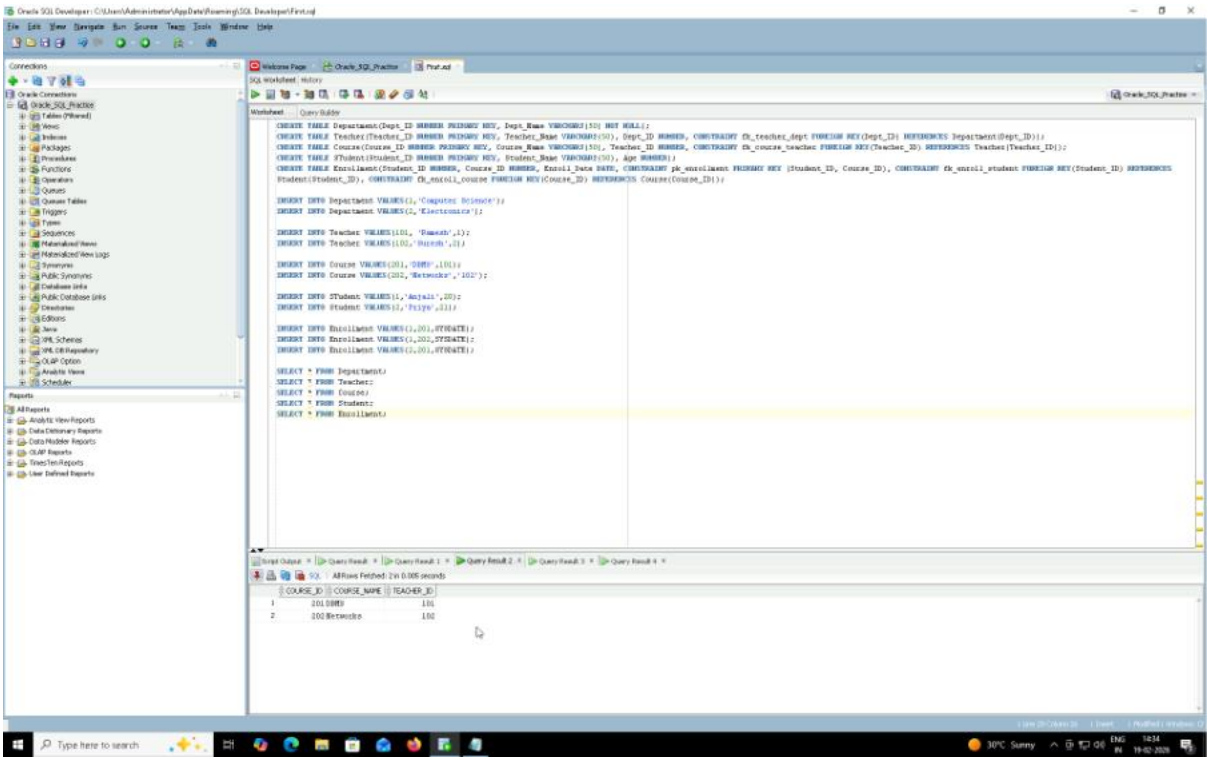
Records have been inserted into Student Table.

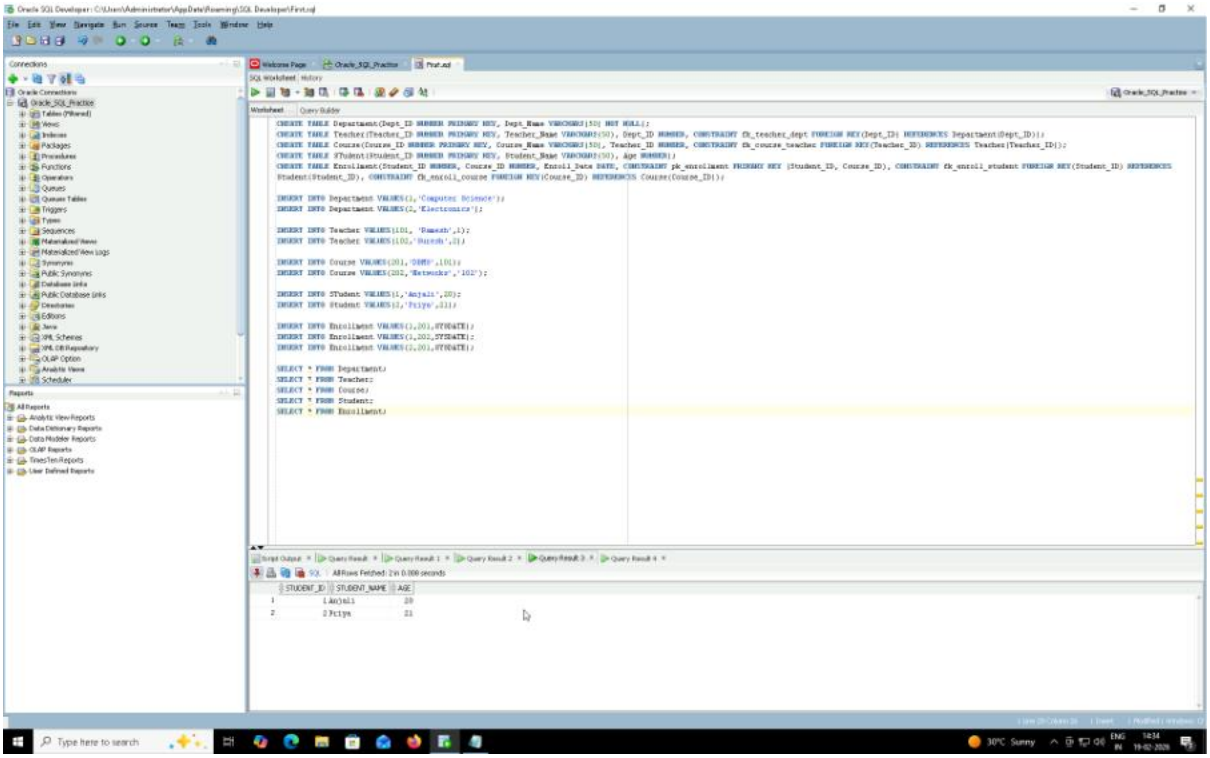


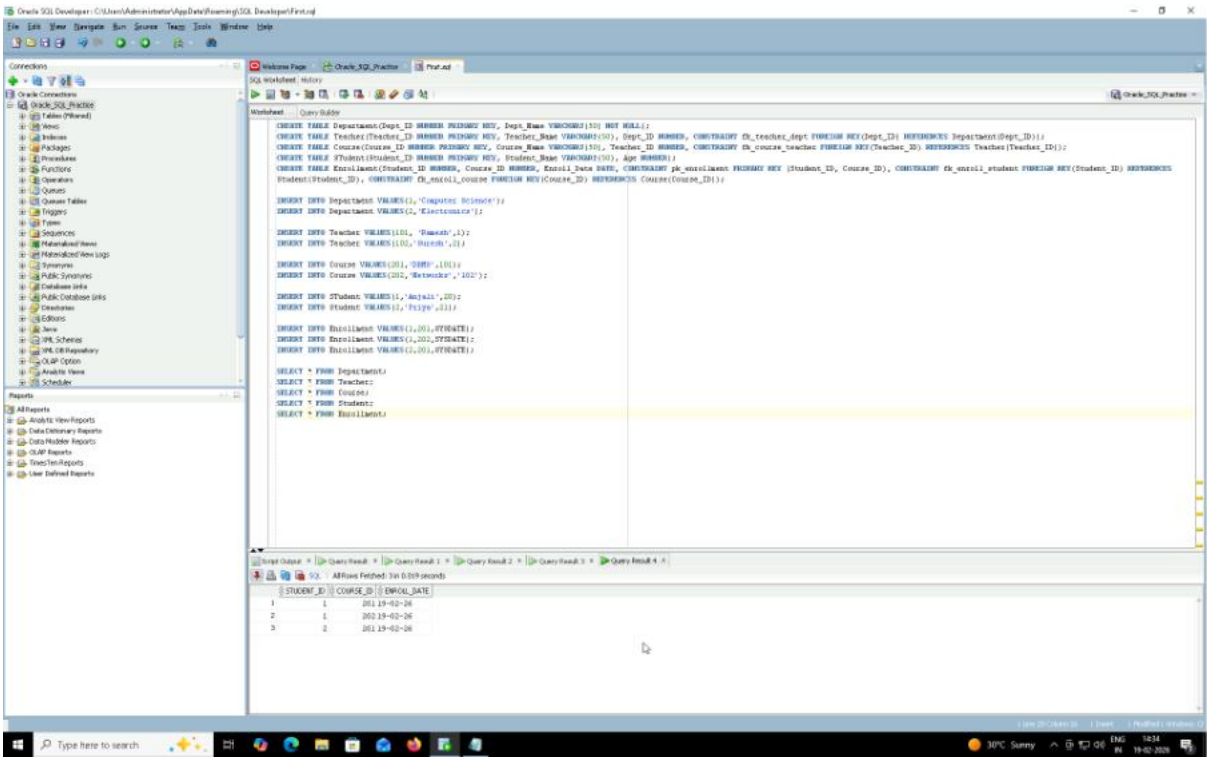
Records have been inserted into Enrollment Table.



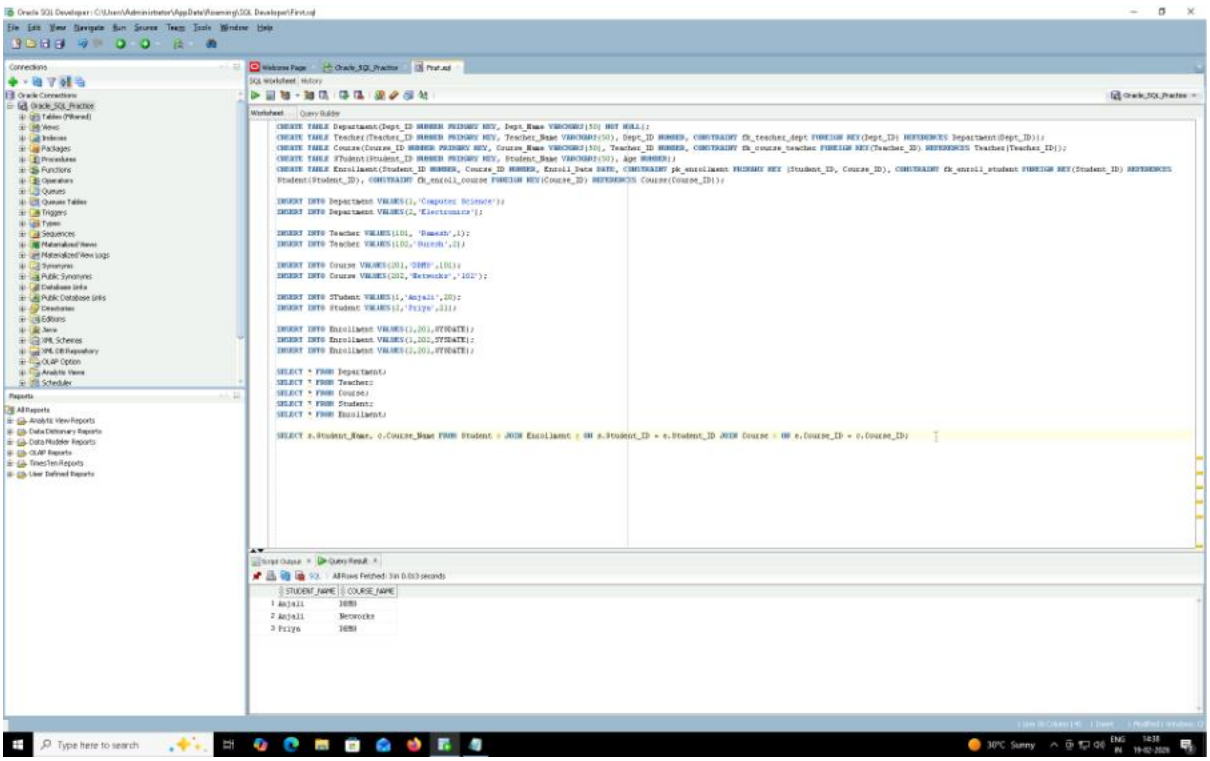

CREATE TABLE Department(Dept_ID NUMBER PRIMARY KEY, Dept_Name VARCHAR2(50) NOT NULL);
CREATE TABLE Teacher(Teacher_ID NUMBER PRIMARY KEY, Teacher_Name VARCHAR2(50), Dept_ID NUMBER, CONSTRAINT fk_teacher_dept FOREIGN KEY(Dept_ID) REFERENCES Department(Dept_ID));
CREATE TABLE Course(Course_ID NUMBER PRIMARY KEY, Course_Name VARCHAR2(50), Teacher_ID NUMBER, CONSTRAINT fk_course_teacher FOREIGN KEY(Teacher_ID) REFERENCES Teacher(Teacher_ID));
CREATE TABLE STudent(Student_ID NUMBER PRIMARY KEY, Student_Name VARCHAR2(50), Age NUMBER);
CREATE TABLE Enrollment(Student_ID NUMBER, Course_ID NUMBER, Enroll_Date DATE, CONSTRAINT pk_enrollment PRIMARY KEY (Student_ID, Course_ID), CONSTRAINT fk_enroll_student FOREIGN KEY(Student_ID) REFERENCES
Student(Student_ID), CONSTRAINT fk_enroll_course FOREIGN KEY(Course_ID) REFERENCES Course(Course_ID));
INSERT INTO Department VALUES(1,'Computer Science');
INSERT INTO Department VALUES(2,'Electronics');
INSERT INTO Teacher VALUES(101, 'Ramesh',1);
INSERT INTO Teacher VALUES(102,'Suresh',2)
INSERT INTO Course VALUES(201,'DBMS',101);
INSERT INTO Course VALUES(202,'Networks','102');
INSERT INTO STudent VALUES(1,'Anjali',20);
INSERT INTO Student VALUES(2,'Priya',21);
INSERT INTO Enrollment VALUES(1,201,07%04%TE);
INSERT INTO Enrollment VALUES(1,202,07%04%TE);
INSERT INTO Enrollment VALUES(2,201,07%04%TE);
SELECT * FROM Department;
SELECT * FROM Teacher;
SELECT * FROM Course;
SELECT * FROM Student;
SELECT * FROM Enrollment;
TEACHER_ID TEACHER_NAME DEPT_ID
1 101 Ramesh 1
2 102 Suresh 2


CREATE TABLE Department(Dept_ID NUMBER PRIMARY KEY, Dept_Name VARCHAR2(50) NOT NULL);
CREATE TABLE Teacher(Teacher_ID NUMBER PRIMARY KEY, Teacher_Name VARCHAR2(50), Dept_ID NUMBER, CONSTRAINT fk_teacher_dept FOREIGN KEY(Dept_ID) REFERENCES Department(Dept_ID));
CREATE TABLE Course(Course_ID NUMBER PRIMARY KEY, Course_Name VARCHAR2(50), Teacher_ID NUMBER, CONSTRAINT fk_course_teacher FOREIGN KEY(Teacher_ID) REFERENCES Teacher(Teacher_ID));
CREATE TABLE STudent(Student_ID NUMBER PRIMARY KEY, Student_Name VARCHAR2(50), Age NUMBER);
CREATE TABLE Enrollment(Student_ID NUMBER, Course_ID NUMBER, Enroll_Date DATE, CONSTRAINT pk_enrollment PRIMARY KEY (Student_ID, Course_ID), CONSTRAINT fk_enroll_student FOREIGN KEY(Student_ID) REFERENCES
Student(Student_ID), CONSTRAINT fk_enroll_course FOREIGN KEY(Course_ID) REFERENCES Course(Course_ID));
INSERT INTO Department VALUES(1,'Computer Science');
INSERT INTO Department VALUES(2,'Electronics');
INSERT INTO Teacher VALUES(101, 'Ramesh',1);
INSERT INTO Teacher VALUES(102,'Suresh',2)
INSERT INTO Course VALUES(201,'DBMS',101);
INSERT INTO Course VALUES(202,'Networks','102');
INSERT INTO STudent VALUES(1,'Anjali',20);
INSERT INTO Student VALUES(2,'Priya',21);
INSERT INTO Enrollment VALUES(1,201,07%04%TE);
INSERT INTO Enrollment VALUES(1,202,07%04%TE);
INSERT INTO Enrollment VALUES(2,201,07%04%TE);
SELECT * FROM Department;
SELECT * FROM Teacher;
SELECT * FROM Course;
SELECT * FROM Student;
SELECT * FROM Enrollment;
COURSE_ID COURSE_NAME TEACHER_ID
1 201 DBMS 101
2 202 Networks 102

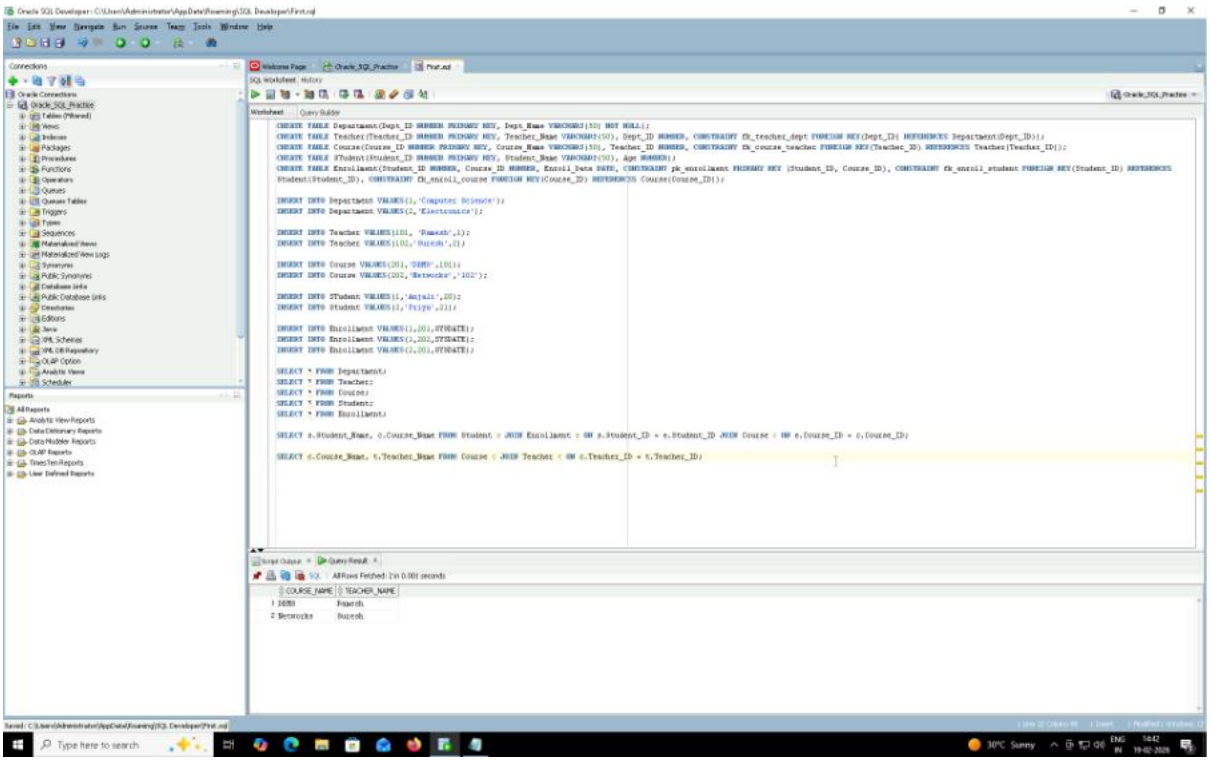




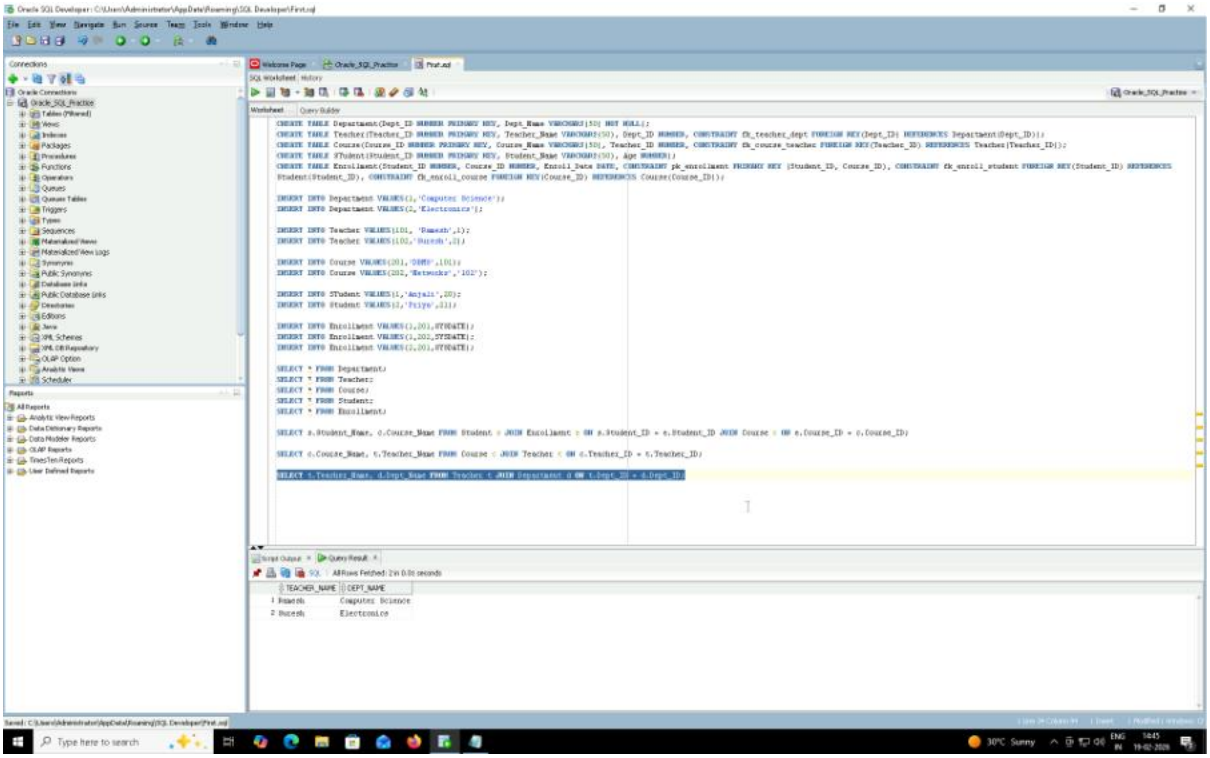
Verified successful insertion of records using SELECT queries.



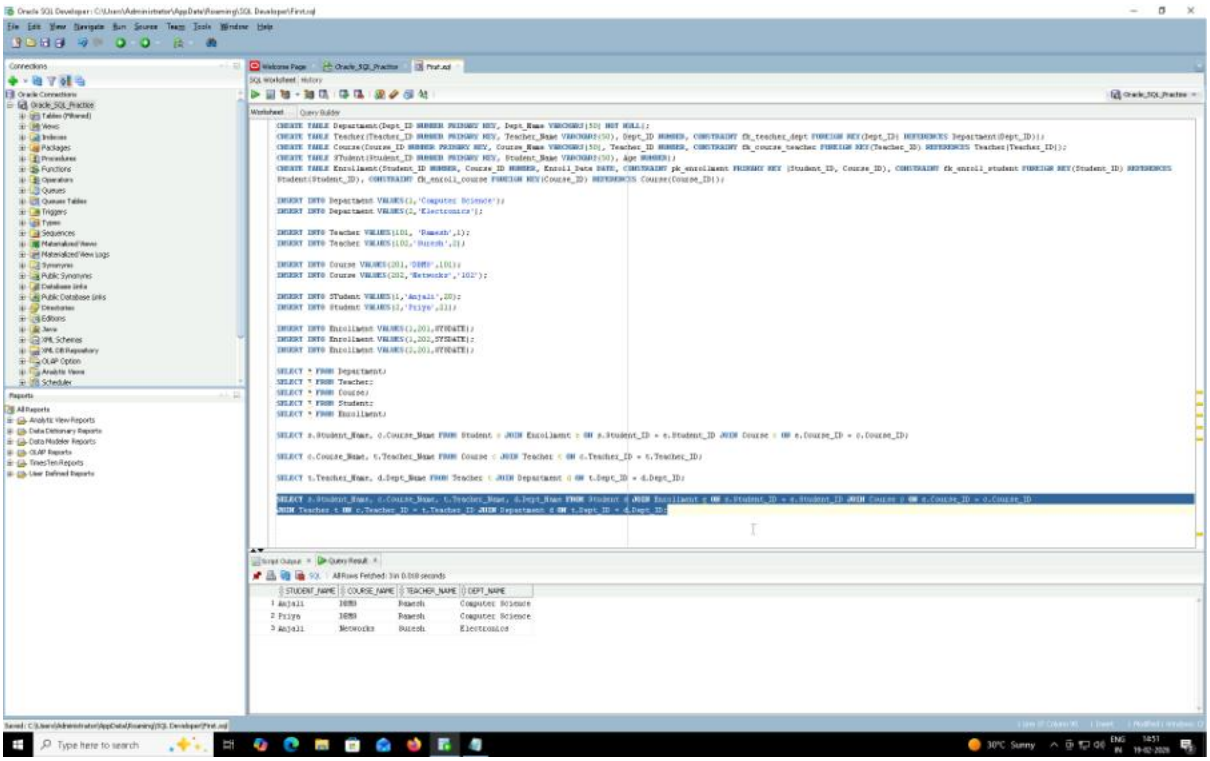
This query verifies the many-to-many relationship between Student and Course through Enrollment table.



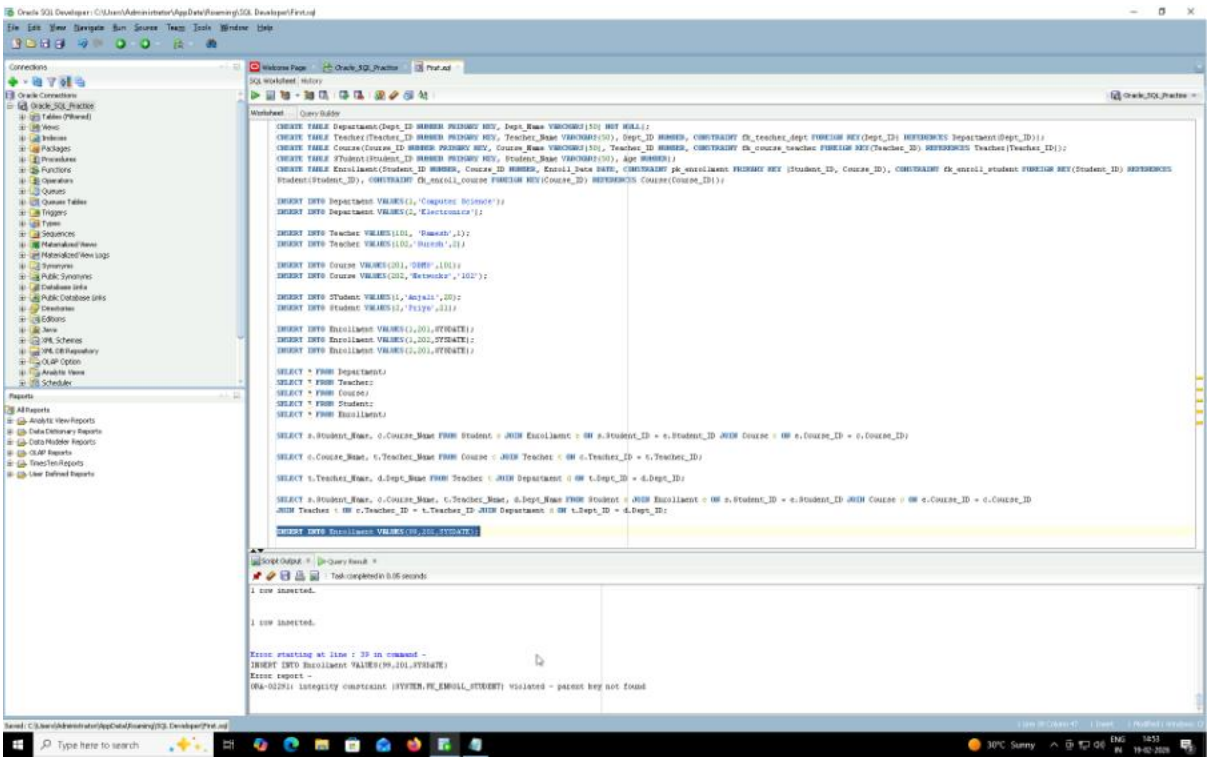
Demonstrates on-to-many relationships between Teacher and Course.



Validates Teacher belongs to a department using foreign key constraint.



This integrated query retrieves complete academic information of a student. It proves the database is normalized and relationships are correctly implemented.



Attempted invalid insertion to test referential integrity. Database rejected the operation, providing foreign key constraints are enforced.

Assignment 2: Using create statement for creating procedures and functions

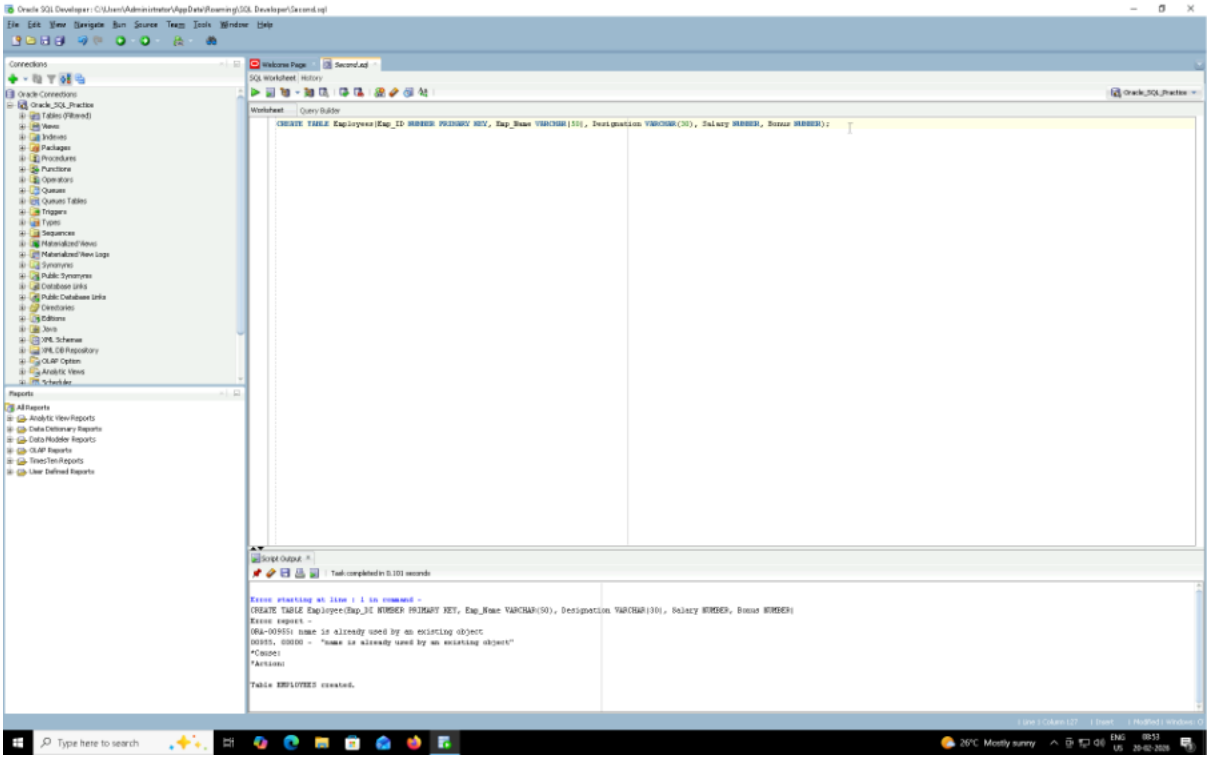
As part of this assignment, you need to complete below tasks:

- Case Study 1: Employee Bonus Calculation (Stored Procedure)
- Objective: Write a stored procedure to calculate and update bonuses for employees based on their designation.
    - Step 1: Create the Employee Table
    - Step 2: Insert Sample Data
    - Step 3: Create the Stored Procedure
- The procedure calculates bonuses based on the designation:
- Manager: 20% of the salary.

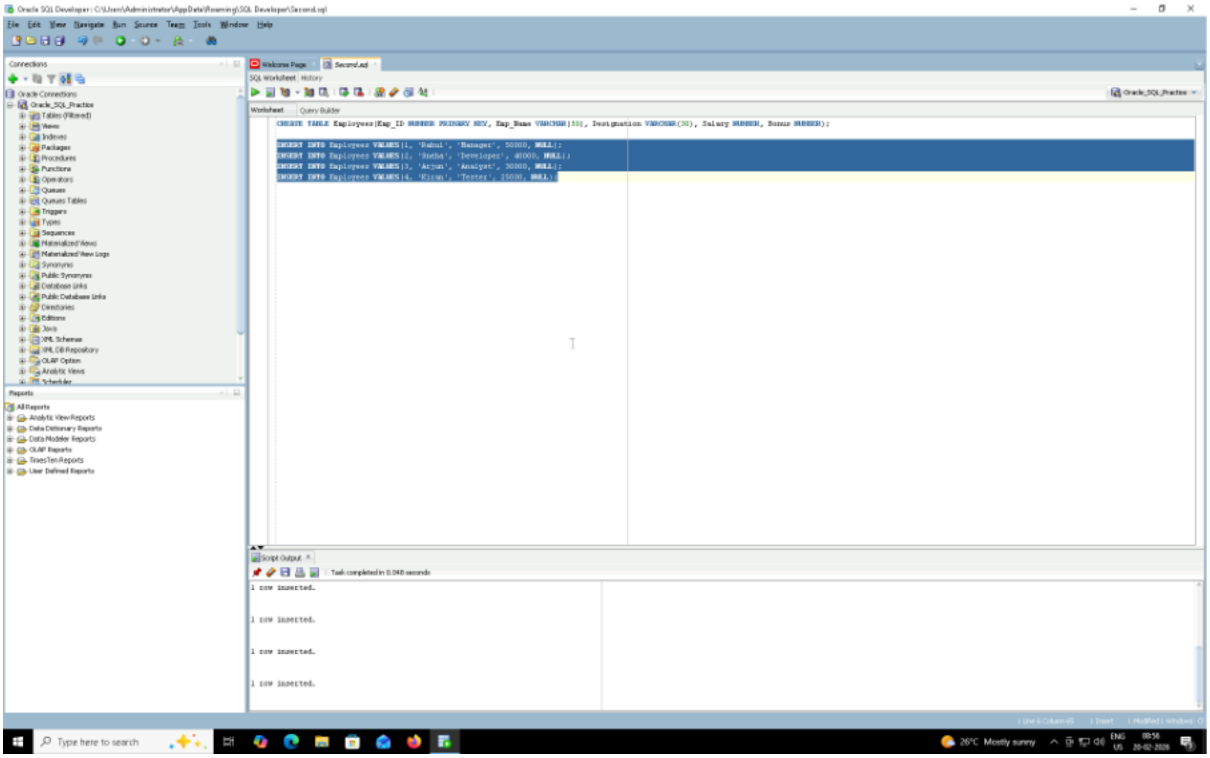
Bonus Given and Assuming: -

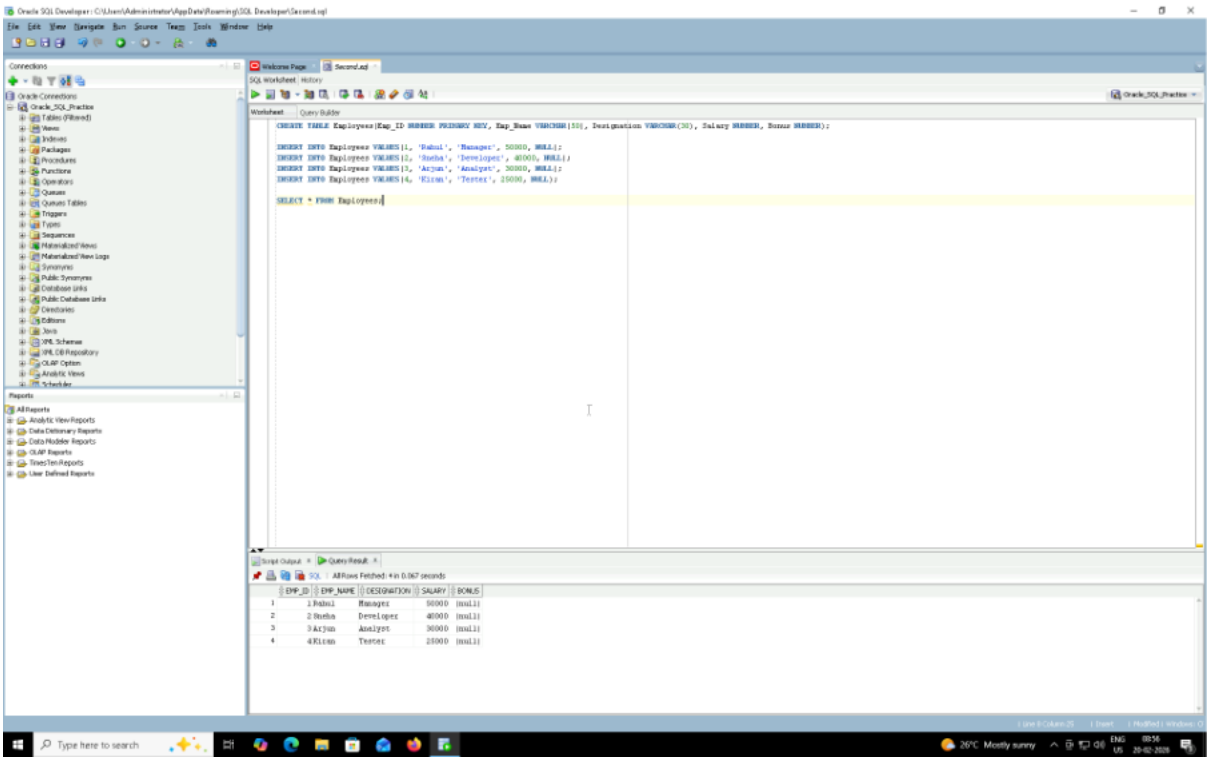
- Manager -> 20% salary
- Developer -> 10% salary

- Analyst -> 5% salary
- Others -> 2% salary

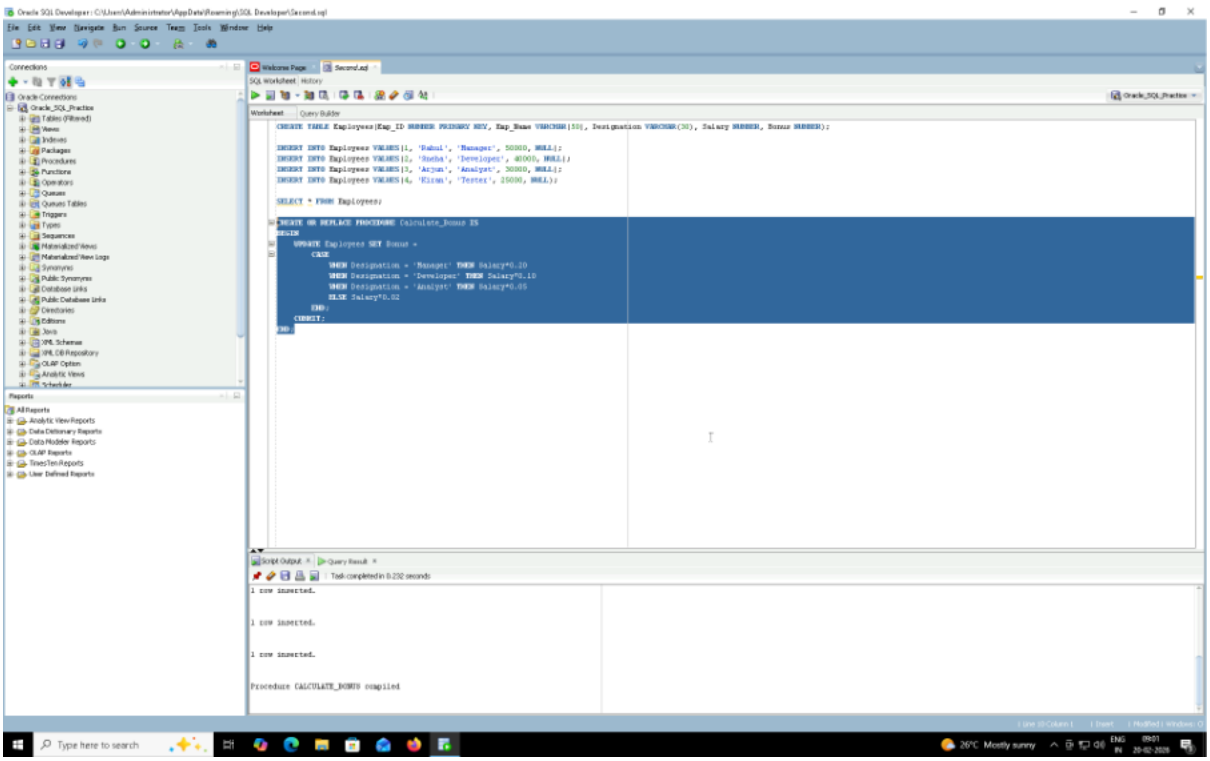


An employees table was created to store details including ID, name, designation, salary and bonus. The primary key constraint was applied on Emp_ID to uniquely identfiy each employee.

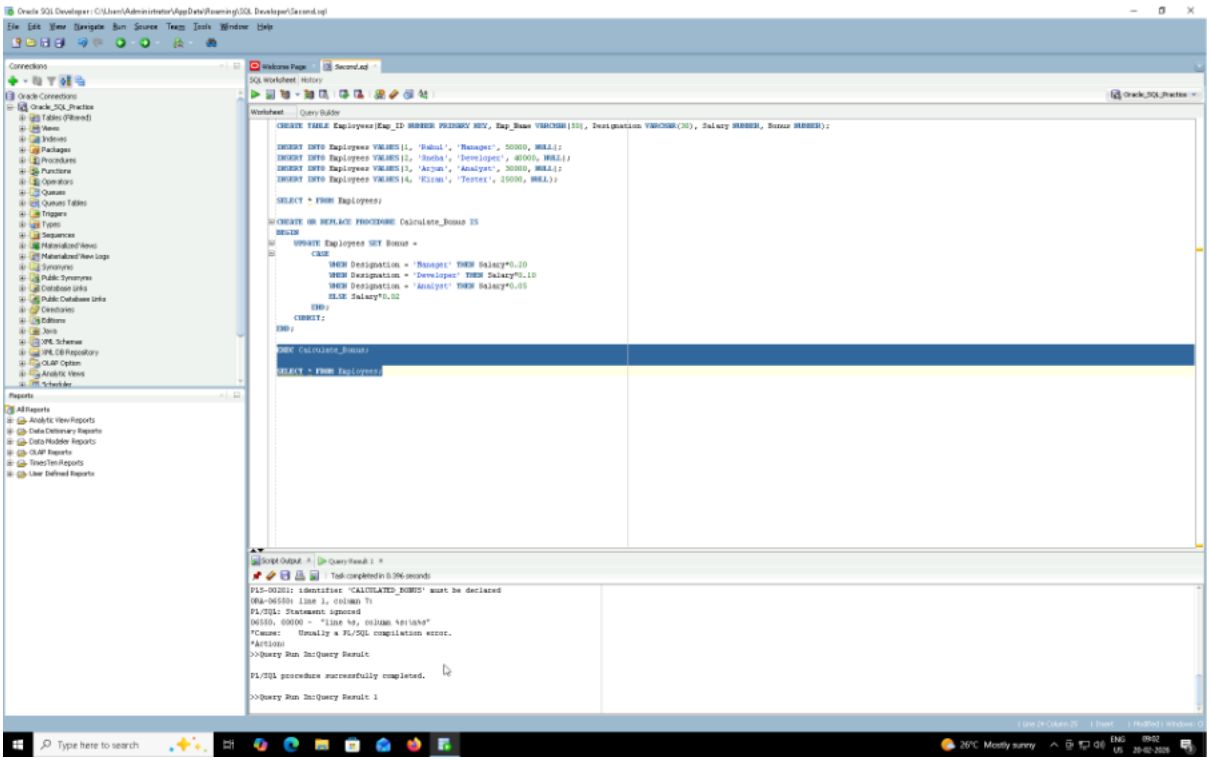




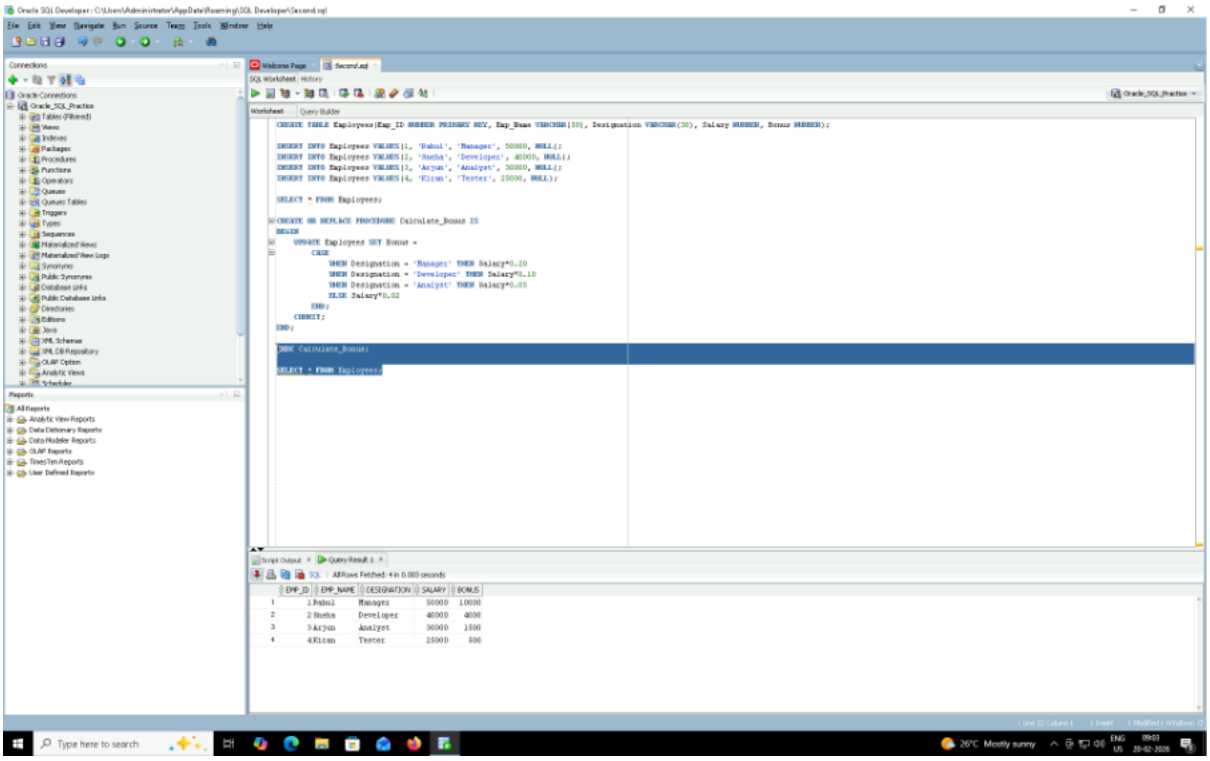
Sample employee records were inserted into the Employees table to test the stored procedure functionality.



A stored procedure name Calculated_Bonus was created. It calculates bonus based on employee designation and updates the Employees table automatically.



The stored procedure was executed successfully, and bonus values were calculated and updated for all employees.



The bonus was correctly calculated according to the designation percentage rules, confirming proper working of the stored procedure.

Conclusion: -

The DBMS assignments proved hands-on experience in designing normalized database schemas and implementing them using Oracle SQL. Concepts such as primary keys, foreign keys, normalization (1NF, 2NF, 3NF), table creation, and referential integrity were successfully applied.

Additionally, stored procedures were developed to automate business logic, demonstrating practical usage of PL/SQL.

These exercises reinforced understanding of relational database design principles and improved skills in database development and structured query execution.